

Complexity of B⁺-Tree Updates and Non-Unique Keys

1 Complexity of B⁺-Tree Updates

1.1 Insertion and Deletion Costs

Although insertion and deletion in B⁺-trees appear complicated, the number of **I/O operations** required is relatively small, which is crucial since I/O is the costliest operation.

Complexity

- Worst-case I/O operations for insertion:

$$O(\log_{\lceil n/2 \rceil}(N))$$

- Worst-case I/O operations for deletion:

$$O(\log_{\lceil n/2 \rceil}(N)) \quad (\text{assuming unique keys})$$

- Here:

- n = maximum number of pointers per node (fanout),
- N = number of records in the file.

- The cost is proportional to the **tree height**, ensuring logarithmic performance.

1.2 Practical Observations

In practice, the actual cost is even lower than the worst-case bounds.

Observations in Practice

- With fanout = 100:
 - Parent of a leaf node is ~ 100 times more likely to be accessed than a leaf.
 - Total non-leaf nodes $\approx 1\%$ of leaf nodes.
 - Non-leaf nodes are often cached in memory $\Rightarrow$ fewer disk I/Os.
- Typically, a lookup requires only **1–2 I/O operations**.
- Probability of a split is low:
 - $\frac{1}{100}$ to $\frac{1}{50}$ insertions result in a split (for fanout = 100).
 - On average, an insertion costs slightly more than **1 I/O**.
- Node occupancy:
 - Random order inserts $\Rightarrow$ nodes are $\sim \frac{2}{3}$ full.
 - Sorted order inserts $\Rightarrow$ nodes are only half full.

1.3 Key Takeaway

Insight

The efficiency of B⁺-trees lies in their ability to keep I/O costs proportional to the tree height, which remains low due to high fanout. This property makes B⁺-trees the **preferred indexing structure** in most database systems.

2 Non-Unique Search Keys

So far, we assumed that search keys are unique. To enforce uniqueness, a **composite search key** can be created by adding a *uniquifier attribute* such as:

- Record ID (pointer),
- Primary key,
- Any attribute unique across all records with the same search key.

2.1 Handling Duplicate Keys

It is possible to allow non-unique search keys in B⁺-trees, but insert, delete, and lookup must be adapted.

Approaches for Non-Unique Keys

1. **Bucket Approach:** Store each key only once in the tree, with a *bucket* (list) of record pointers.
 - Space efficient (key stored once).
 - Complications: variable bucket sizes, buckets larger than a node, or extra I/O if buckets stored separately.
2. **Duplicate-Entry Approach:** Store the key once per record.
 - Splits are handled normally.
 - Higher space overhead (same key stored multiple times).
 - Internal node handling becomes more complex (two leaves may contain the same key).

2.2 Problems with Non-Unique Keys

- **Deletion inefficiency:** Finding the exact record may require scanning multiple leaf entries with the same key $\Rightarrow$ cost can be linear in duplicates.
- Composite unique keys avoid this: deletion requires a single root-to-leaf traversal.

2.3 Practical Implementation

Practical Note

Most database systems avoid supporting duplicate keys directly. Instead, they:

- Automatically add record IDs or other unique attributes,
- Ensure all search keys are unique,
- Guarantee efficient $O(\log N)$ lookup, insertion, and deletion.